

Lección 3.2:

Tipos de mantenimiento de software



¿Por qué importa clasificar el mantenimiento?

La clasificación afecta prioridad, pruebas, responsables y evidencia.

Prioridad

Un fallo crítico puede requerir atención inmediata; una mejora de interfaz puede planificarse.

Impacto

Un cambio adaptativo puede afectar configuración, base de datos, despliegue o ambiente.

Pruebas

Cada tipo exige pruebas diferentes: regresión, rendimiento, compatibilidad o seguridad.

Evidencia

No basta decir “se arregló”: debe existir issue, commit, test, pipeline y versión.

En sistemas con CI/CD, clasificar bien el cambio ayuda a decidir qué debe validar el pipeline antes del despliegue.



Tipos de mantenimiento de software

Todos modifican software ya entregado, pero responden a causas distintas.



Tipos de mantenimiento de software

La categoría depende de la causa principal del cambio, no de la herramienta utilizada.

Pregunta técnica	Tipo probable	Criterio de decisión
¿El sistema hace mal algo que ya debía hacer bien?	Correctivo	Repara un defecto frente a un requisito existente.
¿Cambió el entorno técnico, legal o institucional?	Adaptativo	Permite que el SI siga funcionando en el nuevo contexto.
¿El sistema funciona, pero se busca mejorar valor, rendimiento o usabilidad?	Perfectivo	Mejora la calidad o experiencia actual del producto.
¿Se modifica para evitar fallos futuros, deuda técnica o vulnerabilidades?	Preventivo	Reduce riesgos antes de que afecten al usuario.

Primero se identifica qué cambió y por qué; después se asigna el tipo de mantenimiento.



1. Mantenimiento correctivo

Corrige fallos detectados en el software después de su entrega.

- El sistema no cumple correctamente un requisito ya existente.
- Se origina por errores de código, validación, cálculo, integración o datos.
- La evidencia mínima debe mostrar: bug reportado, causa, corrección y prueba exitosa.

Ejemplo

Error en cálculo de IVA: el total de la factura no coincide con la regla esperada.

Evidencia esperada

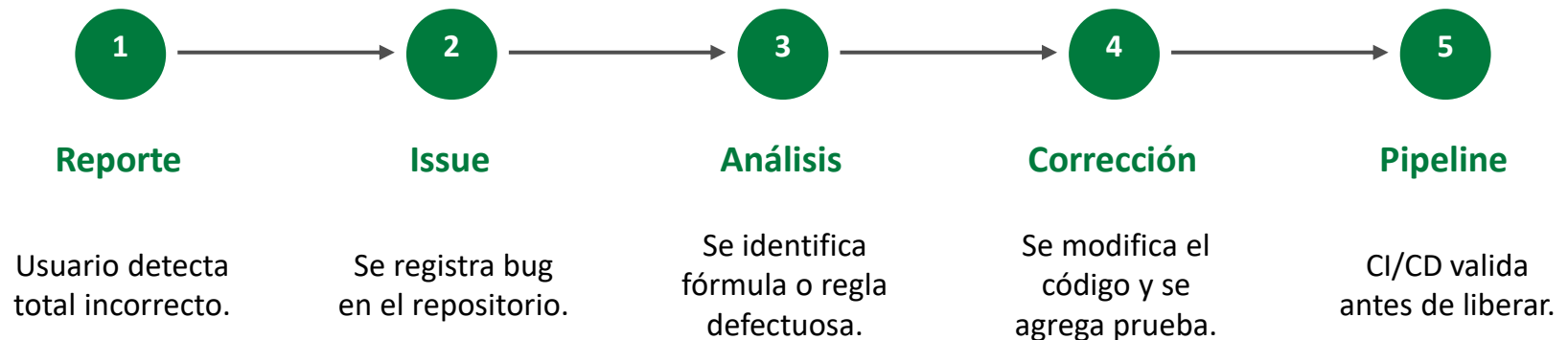
Issue + commit + prueba que falla antes y pasa después + pipeline exitoso.

Si repara un comportamiento incorrecto, es correctivo.



Ejemplo correctivo: error en cálculo de IVA

El sistema ya está entregado, pero genera un resultado incorrecto.



Correctivo, porque corrige un defecto frente a un requisito ya existente: calcular correctamente el total.

2. Mantenimiento adaptativo

Adapta el software a un cambio externo en el entorno técnico, normativo u operativo.

- El sistema no necesariamente está mal; el contexto cambió.
- Puede implicar base de datos, sistema operativo, API, framework, navegador, servidor o normativa.
- Busca que el SI siga funcionando correctamente en el nuevo entorno.

Ejemplo técnico

Migrar de MySQL 5 a MySQL 8 o PostgreSQL manteniendo las funcionalidades existentes.

Ejemplo institucional

Cambian reglas de matrícula; se ajustan validaciones, reportes y mensajes.

Si el cambio externo obliga a modificar el sistema para que siga siendo útil, es adaptativo.



3. Mantenimiento perfecto

Mejora funcionalidad, rendimiento, usabilidad o mantenibilidad del sistema.

- El sistema funciona, pero puede funcionar mejor.
- Puede mejorar experiencia de usuario, velocidad, reportes, navegación o claridad de procesos.
- No necesariamente corrige un bug; aumenta valor o calidad percibida.

Ejemplo

Una consulta de reportes tarda 10 segundos. Se optimiza y ahora responde en 1 segundo.

Evidencia

Antes/después de tiempo de respuesta, prueba de rendimiento o medición documentada.

Si el cambio mejora un sistema que ya funcionaba, normalmente es perfecto.



4. Mantenimiento preventivo

Evita problemas futuros antes de que afecten al usuario o al sistema.

- No espera a que el fallo ocurra en producción.
- Reduce deuda técnica, duplicación, vulnerabilidades latentes o fragilidad del código.
- Suele incluir refactorización, actualización preventiva, pruebas adicionales y limpieza técnica.

Ejemplo

Refactorizar código duplicado antes de que cambios futuros generen inconsistencias.

Evidencia

Código antes/después, reducción de duplicación, pruebas unitarias y pipeline exitoso.

Si se actúa para evitar un fallo probable, es preventivo.



Perfectivo vs preventivo

Ambos mejoran, pero no por la misma razón.

Mejorar hoy

Antes

```
SELECT * FROM pedidos
WHERE estado = 'ACTIVO'
```

 10 segundos

Después

```
SELECT * FROM pedidos
WHERE estado = 'ACTIVO'
```

 1 segundo



Evitar fallos futuros

Antes

```
1 function calcularTotal(pedidos) {
2   let total = 0;
3   for (let i = 0; i < pedidos.length; i++) {
4     total += pedidos[i].importe;
5   }
6   return total;
7 }
8 // Duplicado en otro archivo
```



En el futuro:
bug por cambios
inconsistentes

Después

```
1 function calcularTotal(pedidos) {
2   let total = 0;
3   for (let i = 0; i < pedidos.length; i++) {
4     total += pedidos[i].importe;
5   }
6   return total;
7 }
```



Bug evitado.
Código más limpio
y mantenible.



Caso ambiguo: actualizar una librería vulnerable

La misma acción puede clasificarse distinto según la causa principal.

Escenario A

La librería ya está siendo explotada o causa una falla real en producción.

Correctivo

porque corrige un problema ya presente.

Escenario B

No hay incidente, pero se detecta una vulnerabilidad reportada y se actualiza antes del daño.

Preventivo

porque reduce un riesgo futuro.

La acción técnica es similar; la clasificación depende del motivo del cambio y del estado del problema.



En un SI con CI/CD, la clasificación debe reflejarse en pruebas, trazabilidad y despliegue controlado.

Tipo	Evidencia mínima	Validación esperada
Correctivo	Issue del bug + commit + prueba que falla antes y pasa después.	Prueba funcional/regresión + pipeline exitoso.
Adaptativo	Issue del cambio externo + rama + ajustes de configuración o integración.	Pruebas de compatibilidad, integración, datos o ambiente.
Perfectivo	Medición antes/después + cambio documentado + commit.	Prueba de rendimiento, usabilidad o aceptación del usuario.
Preventivo	Refactorización, actualización o limpieza técnica documentada.	Pruebas de regresión + evidencia de menor riesgo/deuda técnica.

La entrega debe demostrar: solicitud → cambio → prueba → pipeline → versión/despliegue.



Refactoring de método largo

❌ MÉTODO LARGO

Difícil de leer, entender y mantener



Demasiada
lógica junta



Difícil de
leer y seguir



Propenso a
errores

```
1 public void procesarPedido(Pedido pedido) {
2     if (pedido == null) {
3         throw new IllegalArgumentException("Pedido nulo");
4     }
5     if (!pedido.tieneItems()) {
6         throw new IllegalStateException("Pedido sin items");
7     }
8     double total = 0;
9     for (Item item : pedido.getItems()) {
10         if (item.getCantidad() <= 0) {
11             continue;
12         }
13         double precio = item.getPrecio();
14         if (item.tieneDescuento()) {
15             precio = precio - (precio * item.getDescuento() / 100);
16         }
17         total = total + (precio * item.getCantidad());
18     }
19     if (pedido.getCliente() == null) {
20         throw new IllegalStateException("Cliente no definido");
21     }
22     guardarPedido(pedido, total);
23     enviarEmailConfirmacion(pedido.getCliente(), total);
24     if (total > 1000) {
25         aplicarDescuentoEspecial(pedido);
26     }
27 }
```



REFACTORIZACIÓN

Misma lógica,
mejor estructura

✅ MÉTODOS PEQUEÑOS

Código limpio, legible y mantenible



```
1 private void validarPedido(Pedido pedido) {
2     if (pedido == null) {
3         throw new IllegalArgumentException("Pedido nulo");
4     }
5     if (!pedido.tieneItems()) {
6         throw new IllegalStateException("Pedido sin items");
7     }
8 }
```



Responsabilidad
única



```
1 private double calcularTotal(Pedido pedido) {
2     double total = 0;
3     for (Item item : pedido.getItems()) {
4         double precio = calcularPrecioItem(item);
5         total += precio * item.getCantidad();
6     }
7     return total;
8 }
```



Fácil de leer
y probar



```
1 private double calcularPrecioItem(Item item) {
2     double precio = item.getPrecio();
3     if (item.tieneDescuento()) {
4         precio -= precio * item.getDescuento() / 100;
5     }
6     return precio;
7 }
```



Reutilizable
y enfocada



```
1 private void finalizarPedido(Pedido pedido, double total) {
2     guardarPedido(pedido, total);
3     enviarEmailConfirmacion(pedido.getCliente(), total);
4     if (total > 1000) {
5         aplicarDescuentoEspecial(pedido);
6     }
7 }
```



Clara y fácil
de mantener



```
1 public void procesarPedido(Pedido pedido) {
2     validarPedido(pedido);
3     double total = calcularTotal(pedido);
4     finalizarPedido(pedido, total);
5 }
```



Método principal
simple y legible



MÁS LEGIBLE

Fácil de entender
la intención del código



MÁS MANTENIBLE

Cambios locales,
menor riesgo



MÁS TESTEABLE

Funciones pequeñas
son más fáciles de probar



MÁS REUTILIZABLE

La lógica se puede
usar en otros lugares



MENOS ERRORES

Código más simple,
menos bugs



Aplicación al SI del proyecto

Cada grupo debe identificar un cambio real sobre el sistema ya construido.

- Describir la solicitud de cambio o problema detectado.
- Clasificar el tipo de mantenimiento y justificarlo.
- Registrar la solicitud como issue o tarea del proyecto.
- Implementar el cambio en una rama específica.
- Ejecutar pruebas y evidenciar pipeline CI/CD exitoso.
- Documentar la modificación en README, CHANGELOG o reporte breve.

Esto conecta mantenimiento con DevOps: el cambio debe ser controlado, probado y desplegable.

